# Sentinelleai — Audit Report

MagnetaTokenFactory.sol · 46772a0a-8c87-4637-a5e2-c92a3f0a6cd6 · 2026-05-22 16 h 32 min 04 s

## CAUTION 52 / 100 8 consolidated findings

MagnetaTokenFactory.sol presents no confirmed classic reentrancy exploit (the static-sentinel REE-1 flag is a false positive: all state mutations and events precede external calls, and nonReentrant guards all entry points), and the FACT-2 zero-address findings for creator parameters are also false positives — both createStandardForCreator and createAutoLiquidityForCreator contain explicit require(creator != address(0)) guards. However, the contract carries significant SC01 (Access Control) risk: a single EOA owner controls treasury, fee, and crossChainCreator configuration with no timelock or multisig requirement, matching the 2025–2026 pattern of high-loss single-key compromises (IoTeX ioTube, Drift, Resolv). The crossChainCreator relayer path (SC01/SC03) trusts a bare EOA with no source-chain authentication, signed payload, or replay-nonce protection, enabling token forgery under victim addresses if the relayer key is compromised. A treasury DoS vector (SC10) exists where a reverting treasury contract blocks all paid token creation. The ZAD-1 zero-address setter for crossChainCreator is intentional by design but operationally risky. Collectively these issues trigger OWASP SC01, SC03, and SC10 categories and warrant a CAUTION verdict pending multisig/timelock deployment and cross-chain message hardening.

### Severity breakdown

2 HIGH 2 MEDIUM 2 LOW 2 INFO

## Static sentinel pre-scan

Static sentinels matched 4 pattern(s) — 1 HIGH, 2 MEDIUM, 1 LOW.

> **HIGH** SC08 · REE-1 · line 99
>
> **External call before state mutation (classic reentrancy)**
>
> Pattern call.value()() / .call{value:}() followed by state changes — checks-effects-interactions violated.
>
> `(bool success, ) = payable(treasury).call{value: createFee}("");`

> **LOW** SC01 · ZAD-1 · line 60
>
> **Setter accepts zero-address (no validation)**
>
> set*(address _x) functions that write the parameter directly to state can brick the protocol if called with address(0). Add require(_x != address(0)).
>
> `function setCrossChainCreator(address _creator) external onlyOwner { emit CrossChainCreatorUpdated(crossChainCreator, _creator); crossChainCreator = _creator; }`

> **MEDIUM** SC01 · FACT-2 · line 156
>
> **Factory accepts address parameters without zero-address check**
>
> create*/deploy* functions that store `address` parameters (token, recipient, manager) without `require(_x != address(0))` brick the resulting pool — token transfers revert, fees go to the zero address, etc.
>
> `function createStandardForCreator — unchecked: creator`

> **MEDIUM** SC01 · FACT-2 · line 194
>
> **Factory accepts address parameters without zero-address check**
>
> create*/deploy* functions that store `address` parameters (token, recipient, manager) without `require(_x != address(0))` brick the resulting pool — token transfers revert, fees go to the zero address, etc.
>
> `function createAutoLiquidityForCreator — unchecked: creator`

## Consolidated findings

HIGH CVSS 7.5 SC01 — Access Control

### Single-EOA owner controls all critical factory configuration with no timelock or multisig

The factory inherits Ownable2Step but the owner is a single EOA by default. The owner can immediately and atomically change treasury, createFee, and crossChainCreator with no delay, quorum, or role separation. A compromised owner key allows an attacker to redirect all fees to an attacker-controlled treasury, set crossChainCreator to an attacker EOA to forge tokens under victim addresses, and drain any accidentally held ETH via withdraw(). This matches the 2025–2026 pattern of high-loss single-key admin compromises (IoTeX ioTube $4.3M, Drift $285M, Resolv $23M).

```
setCrossChainCreator (line 59), setCreateFee (line 218), setTreasury (line 226), withdraw (line 248); inherited onlyOwner
```

→ Transfer ownership to a 3-of-5 hardware-backed multisig. Enforce a minimum 24–48 hour timelock on treasury, fee, and crossChainCreator changes. Separate the fee-admin role from the cross-chain relayer admin role.

Confirmed by: attacker, historian, developer

HIGH CVSS 7.8 SC01 / SC03 — Access Control / Message Integrity

### Cross-chain relayer path trusts a single EOA with no source-message authentication or replay protection

createStandardForCreator and createAutoLiquidityForCreator authenticate only msg.sender == crossChainCreator. There is no source chain ID, source sender, signed payload, message hash, DVN quorum, or nonce/usedMessages mapping. If the relayer EOA is compromised or becomes malicious, an attacker can call these functions directly to deploy tokens owned by arbitrary victim addresses (recorded in userTokens[victim] and emitting TokenCreated with creator=victim), forge factory provenance for scam tokens, and replay the same creation message to deploy duplicate tokens. This is the off-chain-relayer equivalent of misconfigured LayerZero peer trust.

```
createStandardForCreator (lines 156–187); createAutoLiquidityForCreator (lines 194–215); crossChainCreator state variable
```

→ Route cross-chain calls through a verified on-chain gateway contract rather than a bare EOA. Include source chain ID, source sender, nonce, and a typed structured payload. Store consumed message hashes in a usedMessages mapping and reject replays. Consider requiring quorum or multisig approval for relayed messages.

Confirmed by: attacker, historian

**MEDIUM** CVSS 5.3 SC10 — Denial of Service

### Treasury external call can permanently DoS paid token creation

createStandardToken forwards the fee to treasury via a low-level call and reverts if it fails. If the treasury is set to a contract with a reverting receive/fallback, excessive gas consumption, or any other failure mode, all paid standard-token creation is blocked for all users until the owner changes the treasury. The token is already deployed and state is already updated before the revert unwinds everything, wasting user gas. This is an availability failure controlled entirely by treasury behavior.

```
createStandardToken lines 96–100; setTreasury lines 226–231
```

→ Use a pull-payment pattern: accrue fees in the factory and let treasury withdraw via a separate function. Alternatively, do not revert token creation solely because immediate fee forwarding failed — log the failure and allow a retry or manual sweep.

Confirmed by: attacker, developer

**MEDIUM** CVSS 4.3 SC10 — Denial of Service

### Free AutoLiquidity token creation enables unbounded storage and indexing spam

createAutoLiquidityToken is free (gas only) and appends to both allTokens and userTokens[msg.sender] without any rate limit, per-user cap, or deposit. An attacker can spam-deploy thousands of tokens, bloating allTokens and making getUserTokens(attacker) prohibitively expensive to call on-chain. If crossChainCreator is compromised, the same attack can be targeted at arbitrary victims via createAutoLiquidityForCreator.

```
createAutoLiquidityToken (lines 116–141); createAutoLiquidityForCreator
(lines 194–215); getUserTokens (lines 236–238)
```

→ Add pagination getters for allTokens and userTokens. Consider a nominal anti-spam deposit for free templates, per-block or per-user rate limits, and per-creator quotas on the cross-chain path.

Confirmed by: attacker

**LOW** CVSS 3.1 SC01 — Access Control

### setCrossChainCreator accepts address(0) without a dedicated disable function

The function comment documents address(0) as the intended disable mechanism, and the cross-chain entry points correctly revert when crossChainCreator == address(0). However, using the same setter for both assignment and disablement increases the risk of an operator accidentally disabling the relayer or setting it to zero when intending to update it. There is no two-step confirmation for this privileged role change.

`setCrossChainCreator (lines 57–61)`

→ Separate the setter into setCrossChainCreator(address nonZero) (with require nonzero) and disableCrossChainCreator() to make intent explicit and reduce operator error.

Confirmed by: historian, developer, formalist

**LOW** CVSS 2.5 SC10 — Denial of Service

### withdraw() can be blocked if owner is a contract that cannot receive ETH

withdraw() sends the full ETH balance to owner() via a low-level call and reverts on failure. If ownership is transferred to a multisig, timelock, or contract whose receive function reverts, accidentally sent ETH (e.g., via selfdestruct) becomes permanently stuck until ownership is transferred to a payable address.

`withdraw (lines 248–254)`

→ Allow the owner to specify a payable recipient address in withdraw(address payable to), or use a pull-payment treasury that is known to accept ETH.

Confirmed by: attacker, developer

**INFO** CVSS 0.0 SC08 — Reentrancy (refuted)

### Static-sentinel REE-1 refuted: no classic reentrancy in createStandardToken

The sentinel flagged the treasury call at line 99 as an external call before state mutation. In the actual source, all state mutations (userTokens.push, allTokens.push) and the TokenCreated event occur before the treasury and refund calls. The function is also protected by nonReentrant. A reentrant treasury would be blocked by ReentrancyGuard. The finding is a false positive for classic reentrancy; the treasury DoS risk is reported separately as a SC10 finding.

`createStandardToken (line 99)`

→ No code change required for reentrancy. Retain nonReentrant and current CEI ordering. Address the treasury DoS risk via pull-payment pattern.

Confirmed by: attacker, historian

**INFO** CVSS 0.0 SC01 (refuted)

### Static-sentinel FACT-2 refuted: creator zero-address checks are present in both cross-chain functions

The sentinel reported that createStandardForCreator (line 156) and createAutoLiquidityForCreator (line 194) store creator without zero-address validation. Both functions contain explicit require(creator != address(0), 'Creator cannot be zero') before any token deployment or storage writes. These are false positives.

```
createStandardForCreator (line 169); createAutoLiquidityForCreator (line 204)
```

→ No code change required. Existing guards are sufficient.

Confirmed by: attacker

## Historical precedents

Retrieved from the local bug-bounty corpus (~20 700 findings) by vulnerability-class match.

> **CRITICAL** `reentrancy` ethereum 2022-04-30 **$80 000 000**
>
> Rari Capital/Fei Protocol — Flashloan Attack + Reentrancy (2022-04-30)
>
> https://certik.medium.com/fei-protocol-incident-analysis-8527440696cc

> **CRITICAL** `reentrancy` fantom 2021-12-18 **$30 000 000**
>
> Grim Finance — Flashloan & Reentrancy (2021-12-18)
>
> https://cointelegraph.com/news/defi-protocol-grim-finance-lost-30m-in-5x-reentrancy-hack

> **CRITICAL** `reentrancy` ethereum 2020-04-19 **$25 000 000**
>
> LendfMe — ERC777 Reentrancy (2020-04-19)
>
> https://peckshield.medium.com/uniswap-lendf-me-hacks-root-cause-and-loss-analysis-50f3263dcc09

> **CRITICAL** `access-control` ethereum 2025-05-28 **$12 000 000**
>
> Corkprotocol — Access Control (2025-05-28)
>
> https://x.com/SlowMist_Team/status/1928100756156194955

> **HIGH** `access-control` bsc 2023-03-28 **$8 900 000**
>
> SafeMoon Hack — Access Control (2023-03-28)
>
> https://twitter.com/zokyo_io/status/1641014520041840640

> **HIGH** `access-control` linea 2024-06-01 **$6 880 000**
>
> VeloCore — lack-of-access-control (2024-06-01)
>
> https://x.com/BeosinAlert/status/1797247874528645333

> **LOW** `bm25` 2021-11-01
>
> nested — Missing zero-address check in setters
>
> https://github.com/code-423n4/2021-11-nested-findings/issues/83

> **MEDIUM** `bm25` 2023-07-01
>
> tapioca — MISSING ZERO-ADDRESS CHECK IN CONSTRUCTOR
>
> https://github.com/code-423n4/2023-07-tapioca-findings/issues/81

> **GAS** `bm25` 2021-11-01
>
> nested — Add zero-address checkers
>
> https://github.com/code-423n4/2021-11-nested-findings/issues/108

> **MEDIUM** `bm25` 2023-04-01
>
> frankencoin — `_challengeSeconds ` accepts 0 value when creating a position
>
> https://github.com/code-423n4/2023-04-frankencoin-findings/issues/794

**MEDIUM** bm25 2023-09-01

ondo — No zero address check on constructor parameters in contracts

https://github.com/code-423n4/2023-09-ondo-findings/issues/541

**MEDIUM** bm25 2022-05-01

rubicon — `RubiconMarket.feeTo` set to zero-address can DoS `buy` function

https://github.com/code-423n4/2022-05-rubicon-findings/issues/319

---

Generated by Sentinelleai on 2026-05-22T20:39:33.199Z. Audit ID: `46772a0a-8c87-4637-a5e2-c92a3f0a6cd6`.

This report combines deterministic regex+AST sentinels, optional Slither/Mythril static analysis, and a 6-model adversarial LLM panel consolidated by a Judge agent.